Practical Session 2: Variables, Data Types, and Operators

Learner-Centered, Self-Paced Lab

- **Learning Outcomes Covered**

By the end of this practical session, you will be able to:

1. **Choose** appropriate data types for variables based on problem requirements (e.g., using double vs int for currency).
2. **Predict** the output of expressions involving operator precedence and type conversion.
3. **Analyze** a program to identify potential type mismatch errors before compilation.
4. **Construct** a simple interactive program that accepts user input using Scanner and performs calculations.

Time Required: 90–120 minutes
Instructor Intervention: None – you will figure it out yourself!

- **Before You Begin**

Ensure you have completed **Week 1 practical sessions**. You will need:

- VS Code open
- Your java-course folder with week1 subfolder
- Git configured and connected to GitHub

- **Your Roadmap**



- **TASK 1: Exploring Data Types**

Objective

Choose appropriate data types for different variables based on their purpose.

Step 1.1: Create a New Folder

1. Open **VS Code**.
2. Open your java-course folder.
3. Right-click the java-course folder → **New Folder**.
4. Name it: week2
5. Press Enter.

Step 1.2: Create the File

1. Right-click the week2 folder → **New File**.
2. Name the file: DataTypesDemo.java

3. Press Enter.

Step 1.3: Write the Code

Copy and paste this code into DataTypesDemo.java:

java

```
// This program demonstrates the use of different data types in Java.
// Each variable is declared with the most appropriate data type for its purpose.

public class DataTypesDemo {
    public static void main(String[] args) {

        // ============================================================
        // INTEGER TYPES - used for whole numbers
        // ============================================================

        // byte: 8-bit signed integer, range -128 to 127
        // Used for small numbers to save memory.
        byte studentAge = 20;

        // short: 16-bit signed integer, range -32,768 to 32,767
        // Used for medium-sized numbers.
        short yearOfBirth = 2006;

        // int: 32-bit signed integer, range -2.1 billion to 2.1 billion
        // Most commonly used integer type.
        int population = 54000000; // Kenya's population
```

```
// long: 64-bit signed integer, range very large
// Used for very large numbers. Note the 'L' suffix.
long worldPopulation = 8000000000L;

// ============================================================
// DECIMAL TYPES - used for numbers with decimal points
// ============================================================

// float: 32-bit floating point, ~7 decimal digits precision
// Used for less precise decimal numbers. Note the 'f' suffix.
float interestRate = 7.5f;

// double: 64-bit floating point, ~15 decimal digits precision
// Most commonly used decimal type. No suffix needed.
double accountBalance = 15000.75;

// ============================================================
// CHARACTER TYPE - used for single characters
// ============================================================

// char: 16-bit Unicode character, enclosed in single quotes
char grade = 'A';
char currencySymbol = 'K';

// ============================================================
// BOOLEAN TYPE - used for true/false values
```

```
// ============================================================

// boolean: only two possible values - true or false
boolean isActive = true;
boolean isVerified = false;

// ============================================================
// STRING TYPE - used for text (sequence of characters)
// ============================================================

// String: reference type, enclosed in double quotes
String studentName = "Alice Wanjiku";
String courseName = "Object-Oriented Programming";

// ============================================================
// PRINTING ALL VARIABLES WITH LABELS
// ============================================================

System.out.println("=== INTEGER TYPES ===");
System.out.println("Student Age (byte): " + studentAge);
System.out.println("Year of Birth (short): " + yearOfBirth);
System.out.println("Kenya Population (int): " + population);
System.out.println("World Population (long): " + worldPopulation);

System.out.println("\n=== DECIMAL TYPES ===");
System.out.println("Interest Rate (float): " + interestRate + "%");
```

```
        System.out.println("Account Balance (double): KSh " + accountBalance);

        System.out.println("\n=== CHARACTER TYPE ===");
        System.out.println("Grade (char): " + grade);
        System.out.println("Currency Symbol (char): " + currencySymbol);

        System.out.println("\n=== BOOLEAN TYPE ===");
        System.out.println("Is Active (boolean): " + isActive);
        System.out.println("Is Verified (boolean): " + isVerified);

        System.out.println("\n=== STRING TYPE ===");
        System.out.println("Student Name (String): " + studentName);
        System.out.println("Course Name (String): " + courseName);
    }
}
```

Step 1.4: Run the Program

1. Press Ctrl + S to save.
2. Click the **Run** button (▶) at the top-right of VS Code.
3. Look at the terminal. You should see:

```
text
=== INTEGER TYPES ===
Student Age (byte): 20
Year of Birth (short): 2006
Kenya Population (int): 54000000
World Population (long): 8000000000
```

=== DECIMAL TYPES ===

Interest Rate (float): 7.5%

Account Balance (double): KSh 15000.75

=== CHARACTER TYPE ===

Grade (char): A

Currency Symbol (char): K

=== BOOLEAN TYPE ===

Is Active (boolean): true

Is Verified (boolean): false

=== STRING TYPE ===

Student Name (String): Alice Wanjiku

Course Name (String): Object-Oriented Programming

Step 1.5: Answer These Questions (Write in your notebook)

1. Why did we use long for world population but int for Kenya's population?
2. Why did we use double for account balance instead of float?
3. What is the difference between char and String?
4. What suffix is required for long values?
5. What suffix is required for float values?

✓ **Task 1 Complete**

- □ I created DataTypesDemo.java
- □ I ran the program successfully
- □ I answered the questions

- **TASK 2: Operators and Expressions**

Objective

Predict the output of expressions involving operator precedence and type conversion.

Step 2.1: Create the File

Create a new file called OperatorsDemo.java in the week2 folder.

Step 2.2: Write the Code

Copy and paste this code:

java

```
// This program demonstrates arithmetic, relational, and logical operators.
// It also shows operator precedence and type conversion.


public class OperatorsDemo {
    public static void main(String[] args) {

        // ============================================================
        // ARITHMETIC OPERATORS
        // ============================================================

        // Declare two integer variables.
        int a = 17;
        int b = 5;

        // Addition: adds two numbers.
        int sum = a + b;

        // Subtraction: subtracts second from first.
```

```
int difference = a - b;

// Multiplication: multiplies two numbers.
int product = a * b;

// Division: integer division (decimal part truncated).
int quotient = a / b;

// Modulus: remainder of division.
int remainder = a % b;

System.out.println("=== ARITHMETIC OPERATORS ===");
System.out.println("a = " + a + ", b = " + b);
System.out.println("Sum (a + b): " + sum);
System.out.println("Difference (a - b): " + difference);
System.out.println("Product (a * b): " + product);
System.out.println("Quotient (a / b): " + quotient);
System.out.println("Remainder (a % b): " + remainder);

// ============================================================
// OPERATOR PRECEDENCE
// ============================================================

// Multiplication and division have higher precedence than addition.
// Expression: 10 + 5 * 2
// Step 1: 5 * 2 = 10
```

```
// Step 2: 10 + 10 = 20
int precedence1 = 10 + 5 * 2;

// Parentheses can override precedence.
// Expression: (10 + 5) * 2
// Step 1: 10 + 5 = 15
// Step 2: 15 * 2 = 30
int precedence2 = (10 + 5) * 2;

// Modulus has same precedence as multiplication and division.
// Expression: 20 % 3 + 4 * 2
// Step 1: 20 % 3 = 2
// Step 2: 4 * 2 = 8
// Step 3: 2 + 8 = 10
int precedence3 = 20 % 3 + 4 * 2;

System.out.println("\n=== OPERATOR PRECEDENCE ===");
System.out.println("10 + 5 * 2 = " + precedence1);
System.out.println("(10 + 5) * 2 = " + precedence2);
System.out.println("20 % 3 + 4 * 2 = " + precedence3);

// ============================================================
// RELATIONAL OPERATORS
// ============================================================

// Relational operators return boolean values (true or false).
```

```
boolean isEqual = (a == b);          // Equal to
boolean isNotEqual = (a != b);       // Not equal to
boolean isGreater = (a > b);         // Greater than
boolean isLess = (a < b);            // Less than
boolean isGreaterOrEqual = (a >= b);  // Greater than or equal
boolean isLessOrEqual = (a <= b);     // Less than or equal

System.out.println("\n=== RELATIONAL OPERATORS ===");
System.out.println("a == b: " + isEqual);
System.out.println("a != b: " + isNotEqual);
System.out.println("a > b: " + isGreater);
System.out.println("a < b: " + isLess);
System.out.println("a >= b: " + isGreaterOrEqual);
System.out.println("a <= b: " + isLessOrEqual);

// ============================================================
// LOGICAL OPERATORS
// ============================================================

// Logical operators combine boolean expressions.
boolean x = true;
boolean y = false;

boolean andResult = x && y;   // AND: true only if both are true
boolean orResult = x || y;    // OR: true if at least one is true
boolean notResult = !x;       // NOT: reverses the value
```

```
        System.out.println("\n=== LOGICAL OPERATORS ===");
        System.out.println("x = " + x + ", y = " + y);
        System.out.println("x && y: " + andResult);
        System.out.println("x || y: " + orResult);
        System.out.println("!x: " + notResult);

        // ============================================================
        // INCREMENT AND DECREMENT OPERATORS
        // ============================================================

        int counter = 5;

        // Post-increment: use current value, then increment.
        int postIncrement = counter++;

        // Pre-increment: increment first, then use new value.
        int preIncrement = ++counter;

        System.out.println("\n=== INCREMENT/DECREMENT ===");
        System.out.println("counter after post-increment: " + postIncrement);
        System.out.println("counter after pre-increment: " + preIncrement);
        System.out.println("Final counter value: " + counter);
    }
}
```

Step 2.3: Predict Before Running

Before running the program, write down your predictions for:

1. 10 + 5 * 2 = ?
2. (10 + 5) * 2 = ?
3. 20 % 3 + 4 * 2 = ?
4. a > b (where a=17, b=5) = ?
5. x && y (where x=true, y=false) = ?

Step 2.4: Run the Program

Save and run. Compare your predictions with the actual output.

Step 2.5: Answer These Questions

1. What is the difference between a / b and a % b?
2. Why does 10 + 5 * 2 equal 20 and not 30?
3. What is the difference between counter++ and ++counter?
4. What does the ! operator do?
5. What is the result of true || false?

✓ **Task 2 Complete**

- □ I created OperatorsDemo.java
- □ I made predictions before running
- □ I ran the program successfully
- □ I compared my predictions with actual output

✓ **TASK 3: Type Conversion and Analysis**

Objective

Analyze a program to identify potential type mismatch errors and understand implicit and explicit casting.

Step 3.1: Create the File

Create a new file called TypeConversion.java in the week2 folder.

Step 3.2: Write the Code

Copy and paste this code:

java

```
// This program demonstrates implicit and explicit type conversion.
// It also shows what happens when types don't match.

public class TypeConversion {
    public static void main(String[] args) {

        // ============================================================
        // IMPLICIT CASTING (WIDENING) - automatic
        // ============================================================

        // int to double: smaller type to larger type.
        // No data loss occurs.
        int intValue = 100;
        double doubleValue = intValue; // Automatic conversion

        System.out.println("=== IMPLICIT CASTING ===");
        System.out.println("int value: " + intValue);
        System.out.println("double value (from int): " + doubleValue);

        // int to long: automatic widening.
        long longValue = intValue;
```

```
System.out.println("long value (from int): " + longValue);

// float to double: automatic widening.
float floatValue = 3.14f;
double doubleFromFloat = floatValue;
System.out.println("double value (from float): " + doubleFromFloat);

// ============================================================
// EXPLICIT CASTING (NARROWING) - manual
// ============================================================

// double to int: larger type to smaller type.
// Data loss occurs - decimal part is truncated.
double pi = 3.14159;
int truncatedPi = (int) pi; // Explicit cast with (int)

System.out.println("\n=== EXPLICIT CASTING ===");
System.out.println("double pi: " + pi);
System.out.println("int pi (truncated): " + truncatedPi);

// double to float: explicit cast required.
double largeDouble = 123.456789;
float smallFloat = (float) largeDouble;
System.out.println("double value: " + largeDouble);
System.out.println("float value (cast): " + smallFloat);
```

```
// ============================================================
// TYPE MISMATCH ERRORS
// ============================================================

// The following lines would cause compilation errors if uncommented:

// ERROR 1: Cannot assign double to int without cast
// int wrongInt = 3.14;

// ERROR 2: Cannot assign String to int
// int wrongString = "100";

// ERROR 3: Cannot assign boolean to int
// int wrongBoolean = true;

// ERROR 4: Cannot assign char to String
// String wrongChar = 'A';

System.out.println("\n=== TYPE MISMATCH ERRORS ===");
System.out.println("The following lines would cause compilation errors:");
System.out.println("int wrongInt = 3.14; // double to int without cast");
System.out.println("int wrongString = \"100\"; // String to int");
System.out.println("int wrongBoolean = true; // boolean to int");
System.out.println("String wrongChar = 'A'; // char to String");

// ============================================================
```

```
        // STRING CONVERSION
        // ============================================================

        // Converting numbers to String using concatenation.
        int number = 42;
        String numberString = "" + number; // Convert int to String

        // Converting String to int using Integer.parseInt().
        String numericString = "123";
        int parsedInt = Integer.parseInt(numericString);

        // Converting String to double using Double.parseDouble().
        String decimalString = "45.67";
        double parsedDouble = Double.parseDouble(decimalString);

        System.out.println("\n=== STRING CONVERSION ===");
        System.out.println("Number as string: " + numberString);
        System.out.println("Parsed int: " + parsedInt);
        System.out.println("Parsed double: " + parsedDouble);
    }
}
```

Step 3.3: Run the Program

Save and run. Observe the output.

Step 3.4: Analyze the Errors

For each of the following code snippets, **write down why it would cause an error**:

java

```
// Snippet A
int x = 3.14;

// Snippet B
double y = 100; // Is this an error?

// Snippet C
int z = (int) 3.99;

// Snippet D
String s = 'A';
```

Step 3.5: Answer These Questions

1. What is implicit casting? Give an example.
2. What is explicit casting? Give an example.
3. What happens to the decimal part when you cast a double to an int?
4. How do you convert a String to an int?
5. How do you convert an int to a String?

✓ **Task 3 Complete**

- □ I created TypeConversion.java
- □ I ran the program successfully
- □ I analyzed the error snippets
- □ I answered the questions

- ✓ **TASK 4: Interactive Calculator with Scanner**

Objective

Construct a simple interactive program that accepts user input using Scanner and performs calculations.

Step 4.1: Create the File

Create a new file called InteractiveCalculator.java in the week2 folder.

Step 4.2: Write the Code

Copy and paste this code:

java

```
// This program demonstrates user input using the Scanner class.
// It creates an interactive calculator that performs basic operations.


// Import the Scanner class from the java.util package.
import java.util.Scanner;


public class InteractiveCalculator {
    public static void main(String[] args) {


        // Create a Scanner object to read input from the keyboard.
        // System.in represents the standard input stream (keyboard).
        Scanner scanner = new Scanner(System.in);


        // ============================================================
        // WELCOME MESSAGE
        // ============================================================
```

```
System.out.println("========================================");
System.out.println("     INTERACTIVE CALCULATOR");
System.out.println("========================================");

// ============================================================
// READ USER'S NAME
// ============================================================

// Prompt the user for their name.
System.out.print("Enter your name: ");

// nextLine() reads the entire line including spaces.
String name = scanner.nextLine();

// Greet the user.
System.out.println("\nHello, " + name + "! Let's do some calculations.\n");

// ============================================================
// READ FIRST NUMBER
// ============================================================

System.out.print("Enter first number: ");

// nextDouble() reads a decimal number from the user.
double num1 = scanner.nextDouble();
```

```
// ============================================================
// READ SECOND NUMBER
// ============================================================

System.out.print("Enter second number: ");

// Read the second number.
double num2 = scanner.nextDouble();

// ============================================================
// PERFORM CALCULATIONS
// ============================================================

// Addition
double sum = num1 + num2;

// Subtraction
double difference = num1 - num2;

// Multiplication
double product = num1 * num2;

// Division - check for division by zero
double quotient = 0;
boolean divisionValid = false;
```

```
if (num2 != 0) {
    quotient = num1 / num2;
    divisionValid = true;
}

// Modulus - only for integers
int intNum1 = (int) num1;
int intNum2 = (int) num2;
int remainder = 0;
boolean modulusValid = false;

if (intNum2 != 0) {
    remainder = intNum1 % intNum2;
    modulusValid = true;
}

// ============================================================
// DISPLAY RESULTS
// ============================================================

System.out.println("\n========================================");
System.out.println("          RESULTS FOR " + name.toUpperCase());
System.out.println("========================================");
System.out.println("First Number: " + num1);
System.out.println("Second Number: " + num2);
System.out.println("----------------------------------------");
```

```
System.out.println("Addition: " + num1 + " + " + num2 + " = " + sum);
System.out.println("Subtraction: " + num1 + " - " + num2 + " = " + difference);
System.out.println("Multiplication: " + num1 + " * " + num2 + " = " + product);

// Display division result only if valid.
if (divisionValid) {
   System.out.println("Division: " + num1 + " / " + num2 + " = " + quotient);
} else {
   System.out.println("Division: Cannot divide by zero!");
}

// Display modulus result only if valid.
if (modulusValid) {
   System.out.println("Modulus: " + intNum1 + " % " + intNum2 + " = " + remainder);
} else {
   System.out.println("Modulus: Cannot divide by zero!");
}

System.out.println("=======================================");

// ========================================================
// ADDITIONAL CALCULATIONS
// ========================================================

// Calculate the average of the two numbers.
double average = (num1 + num2) / 2;
```

```
        // Calculate the square of the first number.
        double square = num1 * num1;

        System.out.println("\n=== ADDITIONAL CALCULATIONS ===");
        System.out.println("Average of " + num1 + " and " + num2 + " = " + average);
        System.out.println("Square of " + num1 + " = " + square);

        // ============================================================
        // CLOSE THE SCANNER
        // ============================================================

        // Always close the Scanner to free up resources.
        scanner.close();

        System.out.println("\nThank you for using the calculator, " + name + "!");
    }
}
```

Step 4.3: Run the Program

1. Save the file.
2. Click **Run**.
3. In the terminal, type your name and press Enter.
4. Type the first number and press Enter.
5. Type the second number and press Enter.
6. Observe the results.

Sample Interaction:

```
==========================================
    INTERACTIVE CALCULATOR
==========================================
Enter your name: John Otieno

Hello, John Otieno! Let's do some calculations.

Enter first number: 25
Enter second number: 4

==========================================
         RESULTS FOR JOHN OTIENO
==========================================
First Number: 25.0
Second Number: 4.0
----------------------------------------
Addition: 25.0 + 4.0 = 29.0
Subtraction: 25.0 - 4.0 = 21.0
Multiplication: 25.0 * 4.0 = 100.0
Division: 25.0 / 4.0 = 6.25
Modulus: 25 % 4 = 1
==========================================

=== ADDITIONAL CALCULATIONS ===
Average of 25.0 and 4.0 = 14.5
```

Square of 25.0 = 625.0

Thank you for using the calculator, John Otieno!

Step 4.4: Experiment

Try running the program with different inputs:

1. Enter 0 as the second number. What happens?
2. Enter a negative number. What happens?
3. Enter a decimal number. What happens?

Step 4.5: Answer These Questions

1. What method reads a full line of text including spaces?
2. What method reads a decimal number?
3. Why do we check if num2 != 0 before dividing?
4. What happens if you try to read an integer with nextDouble()?
5. Why do we close the Scanner at the end?

✓ **Task 4 Complete**

- □ I created InteractiveCalculator.java
- □ I ran the program successfully
- □ I experimented with different inputs
- □ I answered the questions

➢ **TASK 5: Push to GitHub**

Objective

Commit all your Week 2 work to GitHub.

Step 5.1: Stage and Commit

1. Open the **Source Control** panel in VS Code (branch icon).
2. You should see all your new files under "Changes".

3. Click the + icon next to each file to stage them.
4. In the message box, type: Week 2: Variables, data types, operators, and Scanner
5. Click the ✓ **Commit** button.

Step 5.2: Push to GitHub

1. Click the **Sync Changes** button (or the ... menu → Push).
2. If prompted, authenticate.

Step 5.3: Verify on GitHub

1. Open your browser and go to https://github.com/YOUR-USERNAME/java-course
2. You should see your week2 folder with all the Java files.
3. Click on a file to view its contents.

✓ **Task 5 Complete**

- □ I staged all files
- □ I committed with a meaningful message
- □ I pushed to GitHub
- □ I verified my code is on GitHub

➢ **Final Self-Assessment Checklist**

Tick each box only when you've verified it:

- □ I created and ran DataTypesDemo.java
- □ I created and ran OperatorsDemo.java
- □ I created and ran TypeConversion.java
- □ I created and ran InteractiveCalculator.java
- □ I can choose appropriate data types for different scenarios
- □ I can predict the output of expressions with operator precedence
- □ I can identify type mismatch errors
- □ I can read user input using Scanner

- □ I pushed all my Week 2 code to GitHub

If you ticked all boxes → YOU ARE A JAVA DEVELOPER!

➢ **Troubleshooting Guide**

Problem	Solution
cannot find symbol: class Scanner	Add import java.util.Scanner; at the top
InputMismatchException	You entered text instead of a number. Use the correct method
nextLine() skipped after nextInt()	Add an extra scanner.nextLine() to consume the newline
ArithmeticException: / by zero	Check for division by zero before dividing
Program doesn't wait for input	Ensure you are using Scanner correctly
File name doesn't match class name	Rename the file to match the public class name

---

➢ **Reflection Questions**

After completing this lab, answer these in your notebook:

1. Which data type surprised you the most? Why?
2. What was the most challenging part of this practical session?
3. What error message did you encounter, and how did you solve it?
4. In your own words, what is the difference between implicit and explicit casting?
5. How will Scanner be useful in your future Java programs?

➢ **What You've Achieved**

You have just:

- Used **all primitive data types** in Java
- Applied **arithmetic, relational, and logical operators**
- Understood **operator precedence**
- Performed **implicit and explicit type casting**
- Built an **interactive calculator** with user input
- Pushed your code to **GitHub**

You are no longer just a student – you are a developer.

➢ **Next Steps**

In Week 3, you will:

1. Learn about **control flow** (if-else, switch)
2. Write programs that **make decisions**
3. Build a **grade calculator** and **menu-driven system**
4. Continue pushing your code to GitHub

Remember: Every professional developer has a GitHub account. Yours is now growing with each week!

➢ **Additional Resources (Self-Help)**

Topic	Resource
Data Types	https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html
Operators	https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html

Topic	Resource
Scanner Class	https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html
Type Conversion	https://docs.oracle.com/javase/specs/jls/se21/html/jls-5.html

END OF PRACTICAL SESSION 2

"The only way to learn a new programming language is by writing programs in it." – Dennis Ritchie

Instructor Note: You should complete this lab independently. I encourage you to experiment with the code, change values, add new variables, and see what happens. The goal is to build confidence in using data types, operators, and user input. Only intervene if you are stuck for more than 30 minutes on the same issue.